Monarch: system architecture

One Go control plane, a Node agent service, isolated sandboxes, MongoDB. Orange and red dashed areas are untrusted; the server decides everything security-relevant.

BROWSER

untrusted client

Home and Build mode
idea -> questions -> plan -> app

IDE
Monaco editor, xterm terminal, agent chat, Source Control

Agent Hub
Studio, Flows, Playground

Settings
model keys, GitHub sync

Live preview iframe
p-<token> host or tunnel URL
separate origin, no IDE cookie

Generated app (end users)
calls its own workflows
with a per-workflow token

Holds no secrets and makes no security decisions: identity comes from the session cookie only.

EDGE

TLS

HTTPS + WSS
Tailscale Funnel or Caddy
(TLS ends here)

GO CONTROL PLANE (ONE BINARY)

the security authority: identity, ownership, policy, side effects

Transport and auth
session cookie (HttpOnly, SameSite), Google and GitHub OAuth, beta code, origin check, X-Jr CSRF header, login and model-call rate limits

Ownership checks on every resource
user comes from the session; sandbox, project, file, run looked up server-side

Builder
questions -> PRD -> design
codegen -> heal -> launch

Stack detector
20+ stacks, ports,
install and start commands

Sandbox lifecycle
limits, idle and max TTL,
disk reaper, orphan cleanup

Files and terminal
os.Root file API (no escapes),
terminal WebSocket per sandbox

Git service
tokenless git in the sandbox;
fetch and push in a separate
credential-only container

GitHub API
repos, branches, PRs, publish,
README, collaborator invite

Key pool and model router
your keys first, per-key limits,
Groq org caps, fallbacks

Preview proxy
p-<token> host -> sandbox port,
or one quick tunnel per app

Agent proxy /agent/*
sets X-Jr-User from the session, adds the internal token,
strips cookies; Node is reachable only through here

Accounts and secrets
users, sign-in identities merged by verified email,
GitHub tokens and model keys sealed with AES-GCM,
never returned to the browser

NODE AGENT SERVICE

loopback only, trusts only Go

IDE chat and edit engine
SEARCH/REPLACE edits, edit guard,
token-budgeted context

Planning mode
plan, review,
tasks, verify,
walkthrough

OpenGAP team
.gitagent agents,
routing, handoff,
sealed hooks

Command policy
model proposes, server decides: deny, ask the user (server-held approval), or allow

Agent Hub runtime
schemas, tools,
guardrails, memory,
versions

Workflow engine
agents, if, set,
HTTP, approvals,
webhooks

Model layer
Groq, OpenAI, Anthropic, Gemini; per agent and per message; your own keys

SSRF guard
public https only, DNS checked, redirects re-checked

Go -> Node: /agent/* with X-Jr-User + internal token
Node -> Go: shell and file sync, as that user

DATA AND EXTERNAL

MongoDB (Atlas)
users, identities,
projects, keys,
agents + versions,
workflows + versions,
runs, memory, API keys,
playground chats

GitHub
OAuth and REST from Go;
git from the git container

Model providers
Groq, OpenAI, Anthropic,
Gemini; from Node and Go

Cloudflare
quick tunnels, one per
app preview (public mode)

control only: start, exec, files, terminal, preview, git

EXECUTION PLANE: CONTAINER ENGINE

Docker locally, rootless Podman + egress firewall + disk pools in the public deployment. Runs untrusted code; controls nothing.

git over https, token only here

Sandbox: your repo
stack image (14 images), --memory,
--cpus, --pids-limit, no-new-privileges,
dropped caps, ports on 127.0.0.1,
no container-engine socket

Sandbox: generated app
Next.js or any Build mode stack,
same limits, own preview token,
calls its workflows by webhook

Git credential container
fresh alpine/git per fetch or push,
cap-drop ALL, scratch GIT_DIR,
repo config, hooks and filters
never read; token only here

Workspaces
one host folder per sandbox,
bind-mounted at /workspace;
saved projects copied to the data dir

Isolation
separate bridge with inter-container traffic off, idle and max lifetimes, per-user and global sandbox caps, per-sandbox disk reaper;
public deployment: rootless Podman in a locked-down WSL distro, nftables blocks private networks, fixed-size storage pools, preflight refuses to start if any control is missing

HOW TO READ THIS

- Security enforcement point (server-side)
- Boundary between services
- Durable data (MongoDB)
- Untrusted: browser input, repo code, model output
- Request or control flow
- Preview traffic and app-to-workflow webhooks

Prompt to live app: browser -> Go builder -> model router -> files -> sandbox -> preview proxy -> iframe.

Monarch in the cloud: deployment and scaling (proposed)

How the same design scales to thousands of concurrent builders. Today Monarch runs on one host; this is the target, not the current deployment.

